Syllabus - ESILV, "Next-Generation AI" theme (Didier Parisot)

Beta · 2026-10-14-students-1, 14 October 2026

Beta: this resource may still change before its session. The site is updated in place; the pack version above identifies what you read.

> Hand-in channel, 8 October 2026: each sub-group keeps one public Git repository and hands in its link through Moodle after each session; see Hand-in. The mention of the school's GitHub below predates this decision.

> This is the English delivery version; the module is taught in English. French reference: `syllabus-esilv-next-gen-ai.md` (teaching-team source, outside the student pack). Both versions carry the same substance, section by section.
>
> Frame agreed with Nédra Mellouli, head of the Data & AI major: `administratif/knowledge/cr-nedra-mellouli.md` (teaching-team source, outside the student pack), `administratif/knowledge/cr-co-intervenants-2026-09-23.md` (teaching-team source, outside the student pack). Revision of 05/10/2026.

Module title

Trustworthy Generative AI Subtitle: choosing the model up to each question at the lowest cost, and cross-checking several models when the answer carries consequences.

Frame

- ESILV, Data & AI major, **year 4 (S7)**, Next-Generation AI module (MESIIN475926, 2 ECTS), September to December 2026.
- Audience: engineering students taking their first artificial intelligence courses. Close supervision.
- Three days: **21 October, 18 November and 11 December 2026**. On 21 October, the on-site lecture runs from 8:30 to 11:30 and the lab from 14:00 to 18:30, as confirmed on 14 October. The other two lectures are online; use the school timetable for those sessions.
- About 200 students in seven lab groups. Each group of 32 works in four sub-groups of eight, with one instructor per group.
- **Language: the module is taught in English** (lectures and lab sessions), for the international cohort.

Guiding thread

A latest-generation language model answers most well-posed questions correctly. Its price per query and the rationing of its access rule out using it on every one of a company's requests. Hundreds of cheaper models remain available, with more frequent errors. The module teaches the two skills that follow from this situation:

1. **assign each request to the cheapest model that remains up to its difficulty**, through an automatic rule, transparent to the end user;
2. **when the answer carries consequences** (health, investment, legal decision, amount paid out), **query several models and retain only a majority** (k-of-n quorum), with abstention without a majority and independent checking of the result.

The course is taught by a practitioner, from AI systems built and run in production, as a complement to academic teaching.

Guiding case: an assistant that queries the data of an online shop. A SQLite database of three tables (products, customers, orders) and twenty revenue questions are shared by all sub-groups. Each model queried returns a SQL query, executed read-only; the value obtained is compared with a reference computed by the sub-group. Five free models (Groq, OpenRouter) are used for the calls; the real portfolio of a small company (sixty model and access pairs, sorted into five capability groups) is used to reason about cost.

Learning objectives (by the end of the module, the student can)

1. **Read a model portfolio**: capability (public index), price per million tokens in input and in output, access route (subscription with a quota, pay-per-token API access, free plan), and deduce which questions a model is suited to.
2. **Choose, for a request, the cheapest model that remains up to its difficulty**, then write the routing rule that applies this choice automatically, without intervention by the end user.
3. **Check a model's answer against an independently computed reference**, and attribute a discrepancy to the model rather than to the wording of the question, the call parameters or the access.
4. **Query several models on the same question and apply a k-of-n quorum**: retain the majority, abstain without a majority, recognise a wrong consensus.
5. **Quantify a configuration** (single model, quorum, routing) in calls and in tokens with the guided method on free models, then **choose the configuration of a named use case** according to what an error costs there.
6. **Situate a system within agentic orchestration protocols (MCP, Model Context Protocol)**: what they standardise and when a production system benefits from them.

Lectures (three mornings)

Lecture 1, 21 October, on site: models by the hundreds, a finite budget. Level survey at the opening (Python, SQL, prior exposure to language models), to calibrate supervision. The origin of the module: a model output is a draw, the same question returns several answers, a wrong figure looks like a right one. What has changed: latest-generation models are rarely wrong on a well-posed question. What this certainty costs: price per million tokens, cost of a complex processing run, annual bill of a team that systematically queries the most powerful model. Subscription with a quota and pay-per-token API access: two access routes to the same model, two economies. The portfolio: several hundred routes across providers, five capability groups, public index and price side by side. The two decisions of a service: routing by difficulty, cross-checking several models when the answer carries consequences. Presentation of the guiding case, the deliverables and the grid.

Lecture 2, 18 November, online: the quorum and routing. Reading of the TD 1 results across the seven groups. The k-of-n quorum: definition, choice of k, abstention without a majority, cost multiplied by n. Wrong consensus: similar models are wrong together; the quorum exposes disagreement, it does not create truth; a false consensus is more dangerous than a visible disagreement. Complements: field-by-field agreement, escalation on disagreement, limits of a judge model. The routing rule: difficulty, stakes and deadline of the request, eligible models, cheapest available, deferral when the quota is exhausted and the request can wait, escalation or abstention otherwise. What a trace keeps to make these decisions verifiable.

Lecture 3, 11 December, online: cost and decision. Reading of the TD 2 results. What weighs in an inference bill: long context, retries, oversized model, systematic quorum. Capacity management: subscriptions, quotas that refill, buying tokens as a last resort. Cost method on free models: tokens observed in the traces multiplied by the dated price of a paid route of the same model, real spend kept separate. Three use cases, three configurations: internal dashboard, figure shown to the customer, health, investment or legal decision. Experience report: a small company's portfolio and its assignment rule; a multi-model consensus system, shut down when standard models were sufficient. Awareness of agentic orchestration protocols (MCP), distinct from the measurement protocol of the lab sessions. What is expected in the presentation.

Lab sessions (three afternoons)

TD 1, 21 October: comparing models on the same questions. Check of the environment set up before the session. The twenty reference queries, split among the eight members and checked with the instructor. The five authorised models receive the same prompt on the twenty questions; each returned query is executed read-only and compared with the reference, one trace line per call. Reading the traces: which model fails on which questions, for what cause. Each model is situated in the portfolio: capability, price, access.

TD 2, 18 November: the quorum, then the routing rule. Implementation of the k-of-n vote in the skeleton, run on the TD 1 traces with a panel of three model families. Explain at least one identified wrong consensus and its cause, using a supplied historical or synthetic case if the batch contains none. Writing of the sub-group's routing rule: for each question family, the cheapest model up to it according to TD 1, and the cases that call for the quorum. The rule is run on a supplied resource simulator (quota exhausted, deferrable request, urgency, budget exhausted, high stakes).

TD 3, 11 December: quantifying and deciding. Configuration table: single model, quorum, routing; for each, answers returned, abstentions, errors, calls, tokens, API-equivalent cost and real spend. Choice of a configuration for a named use case, justified by what an error costs there. Three-slide presentation; 20-minute oral presentation per sub-group (proposal, school approval pending).

Supervision note: assess the observations and their explanation. The quorum exposes disagreement and can share a wrong assumption. Twenty questions establish neither a general error rate, a permanent ranking nor an error bound guaranteed by quorum.

Deliverables and assessment

Each sub-group deposits on the school's GitHub:

- `reference-td1.csv`: the twenty reference queries and their values;
- the trace tables of the three sessions, all attempts retained;
- `model-question-comparison-en.csv` (long format, matrix displayed by pivot) and `five-model-summary-en.csv`;
- the implemented k-of-n vote, and at least one wrong consensus identified with its cause;
- the sub-group's routing rule, one page, with the simulator outputs;
- the table of configurations with their costs;
- three slides: the models compared and what the traces show; the quorum and the wrong consensus; the recommended configuration for a named use case, with its cost;
- the prompts used, verbatim;
- a recorded video of the demonstration.

Scale proposed to the school (validation in progress):

What is marked	Points
Three observable outcomes in TD 1: reference, traces, cause of a failure	6
Three observable outcomes in TD 2: vote, wrong consensus, routing rule	6
Three observable outcomes in TD 3: costs, decision, limits	6
Hand-in: a repository that can be replayed, a video of the chain	2
Total	**20**

Each outcome is marked 0, 1 or 2 on what the sub-group has shown during the session; the three levels are written on the sheet laid on its table from TD 1. The mark is the sum of the levels: no range, no half point. The oral presentation then adjusts the total by -1 to +1 point. An outcome marked 2 whose evidence is absent from the final repository drops to 1; none rises after the session, which avoids marking the same work twice. Sub-group mark, identical for the eight members.

The prompts are attached and marked: working with the models is required, not merely tolerated. How the system behaves is marked by the voting outcome, 2 points out of 20; reading and criticising code by the two cause-of-failure outcomes, 4 points out of 20. Programming style carries no point in this proposal; the school says whether it should.

Model access, prerequisites, tools

- Python 3.10 or later, SQLite, an editor. No model-architecture prerequisite. SQL basics (`SELECT`, `WHERE`, `SUM`, join) are used for the reference queries; the conventions to apply are written in the protocol.
- Model access: one Groq account (Free plan) and one OpenRouter account per student, with no provisioning by the school; five free models selected by measurement on the twenty questions, an optional sixth. Finish preparation by 18 October inclusive with the kit guide and saved `check_access.py` evidence. The TD1 base batch is 100 cells (60 Groq, 40 OpenRouter), split among five members including two distinct personal OpenRouter accounts. Controls and retries are separate; each account checks its capacity and reserve, with fallback under the protocol.
- The portfolio of sixty model and access pairs is supplied for reading and computation; the lab session calls are made only to the authorised free models.

Coding assistance is proposed with declared use, verification and explanation, pending school approval. The reference is obtained by executing SQL on the database and independently checking it, never by adopting an LLM amount. For costing, an undocumented exact-model paid counterpart or missing token measure gives N/A; a known subtotal is not a complete total.

Next step, outside the lab sessions

Estimating an error rate per model (repeated questions, independent sample, intervals, version tracking) is presented in lecture 3 as the next step of a production system. It is neither taught nor marked in this module.

Possible extension (S9, "Trusted AI" module)

A second part on the **technical side**: knowing and bounding the error margin of a system. **Not** governance, ethics or regulation, which are covered elsewhere.